

Explicação de Código Java

Análise técnica — `CopyOnWriteList<T>`

1. Objetivo Geral

A classe `CopyOnWriteList<T>` implementa uma lista genérica thread-safe baseada no padrão *copy-on-write*: a cada escrita, uma cópia completa da lista interna é criada, modificada e depois publicada atomicamente. O objetivo é garantir que leituras concorrentes nunca bloqueiem e ocorram sempre sobre um estado consistente da estrutura.

2. Lógica e Algoritmo

O método `add(T item)` adquire um `ReentrantLock`, clona a lista atual em um novo `ArrayList`, adiciona o elemento à cópia e reatribui a referência do campo `inner`. A variável `inner` é declarada `volatile`, garantindo visibilidade imediata da nova referência a todas as threads. Leituras (`get` e `size`) acessam `inner` diretamente, sem lock, pois operam sobre a snapshot imutável vigente no momento do acesso.

3. Conceitos Java Envolvidos

Generics (T) para tipagem parametrizada; **volatile** para garantia de visibilidade entre threads sem sincronização explícita na leitura; **ReentrantLock** do pacote `java.util.concurrent.locks` para exclusão mútua nas escritas; **Thread** e **join()** para concorrência básica; e o padrão estrutural *Copy-On-Write*, também usado na JDK em `CopyOnWriteArrayList`.

4. Complexidade

A operação `add` tem complexidade **O(n)** em tempo e espaço, pois copia toda a lista a cada inserção. As operações `get` e `size` são **O(1)**. O padrão é eficiente apenas em cenários com poucas escritas e muitas leituras simultâneas — caso contrário, o custo de cópia torna-o inviável.

5. Decisões de Design e Limitações

O uso de **volatile** + lock apenas na escrita é uma escolha intencional: reads nunca bloqueiam, eliminando contenção de leitura. A principal limitação é o custo **O(n)** por escrita. Além disso, a thread leitora no `main` pode observar tamanho zero se executar antes das escritas, evidenciando que o padrão garante consistência por snapshot, não linearizabilidade entre writer e reader.